

Social Network

Software Design Specification

Date: April 24, 2026

Group Members:

Yunus Kaya
Muhammed Emin Erdoğan
Ubeydullah Tuna
Abdullah Puskar

Prepared for

CSE3044 Software Engineering Term Project

Table of Contents

Table of Contents	2
1. Introduction	3
1.1 Purpose	3
1.2 Statement of Scope	3
1.3 Software Context	3
1.4 Major Constraints	3
1.5 Definitions	4
1.6 Acronyms and Abbreviations	4
1.7 References	4
2. Design Consideration	5
2.1 Design Assumptions and Dependencies	5
2.2 General Constraints	5
2.3 System Environment	5
2.4 Development Methods	5
3. Architectural and Component-Level Design	7
3.1 System Structure	7
3.1.1 Architecture Diagram	7
3.2 Description for Components	7
3.2.1 Component 1: Account & Profile Management	7
3.2.2 Component 2: Content Management	7
3.2.3 Component 3: Recommendation Engine	8
3.2.4 Component 4: Messaging & Notifications	8
3.3 Component Processing Detail & Design Class Hierarchy	9
3.4 Dynamic Behavior	10
4. Restrictions, Limitations, and Constraints	11
5. Conclusion	12
Appendix A – Design Class Diagram (DCD)	13
Appendix B – Sequence Diagrams	14
B.1 - Register with Interest Selection	14
B.2 - Content Recommendation	15
B.3 - Direct Messaging	16

1. Introduction

This section is repeated from the Software Requirements Specification document and provides background information about the project.

1.1 Purpose

This Software Design Specification (DSD) document provides a comprehensive architectural and component-level design for the Social Network system. It describes the data, functional, and behavioral requirements translated into software components, guiding the development team through implementation.

1.2 Statement of Scope

The Social Network is a standalone, web-based social media platform featuring a RESTful API and a lightweight HTML/CSS interface. The scope is classified as follows:

Essential Requirements:

- User registration with interest selection
- JWT-based authentication
- Profile management
- Following/unfollowing users
- Creating posts (text and optional images)

Desirable Requirements:

- Personalized feed generation
- Direct messaging
- Like/comment functionality
- Notification alerts

Future / Out-of-Scope Requirements:

- Real-time messaging (WebSockets)
- Video uploads
- Machine learning-based recommendations
- Third-party authentication

Note on Scope Compliance: The Recommendation Engine described in Section 3.2.3 uses a rule-based scoring approach, which is within scope. However, the sequence diagram (figure B2) introduces a dedicated Recommendation Engine component that communicates with the database independently. This architectural detail was not fully specified in the SRS and should be validated with the project stakeholders.

1.3 Software Context

The software serves as a self-contained web application operating on a client-server architecture. The backend exposes a REST API via Node.js/Express.js, while the frontend consumes these endpoints using standard HTTP requests. A relational database (PostgreSQL) managed via Prisma ORM maintains persistent data storage.

1.4 Major Constraints

- The UI must be implemented in plain HTML/CSS without JavaScript frameworks.
- Server-side logic is strictly confined to Node.js and Express.js.
- Authentication relies entirely on stateless JWT tokens.

- Images must be stored locally on the server filesystem; external cloud storage is prohibited.

1.5 Definitions

Term	Definition
Feed	A personalized list of posts from followed users, sorted by recency.
Recommendation	A post suggested to the user based on rule-based scoring of their interests and interaction history.
Hashtag	A keyword prefixed with # used to categorize post content.
Interest	A topic category selected by the user during registration (e.g., Technology, Sports).
DM	Direct Message – a private text message sent between two users.
Notification	An alert triggered when another user interacts with one's content.

1.6 Acronyms and Abbreviations

Acronym	Meaning
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
ORM	Object-Relational Mapping
SRS	Software Requirements Specification
DSD	Design Specification Document
MVC	Model-View-Controller
OOD	Object-Oriented Design

1.7 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- CSE3044 Software Engineering Term Project Guidelines.
- Social Network – Software Requirements Specification (SRS), March 27, 2026.

2. Design Consideration

This section describes the issues which need to be addressed or resolved before attempting to devise a complete design solution.

2.1 Design Assumptions and Dependencies

- A Node.js-compatible server environment and a configured relational database (PostgreSQL) are available before system initialization.
- The host server filesystem possesses sufficient capacity for image uploads.
- Client browsers will correctly store and attach the JWT token in the Authorization header for protected API routes.
- The interest-based recommendation uses a simple scoring rule: posts whose hashtags match the user's interest list or past interactions are ranked higher. No machine learning is applied.

2.2 General Constraints

Constraint	Description
Performance	The system must support at least 50 concurrent users and respond to 95% of API requests within 500 milliseconds.
Security	Passwords must be hashed using bcrypt, and database queries must use parameterized statements to prevent SQL injection.
Messaging Delivery	Direct messaging is REST-based; users must refresh to see new messages (no WebSockets).
UI Technology	The user interface must be implemented in plain HTML/CSS without any JavaScript framework.
Storage	Image uploads are stored on the server filesystem; external storage services are not used.

2.3 System Environment

Component	Technology
Runtime / Framework	Node.js, Express.js
Database Engine	PostgreSQL
Database Management	Prisma ORM
File Upload Middleware	Multer (multipart/form-data)
Authentication	JSON Web Token (JWT)
Frontend	Plain HTML/CSS (no JS framework)

2.4 Development Methods

The project utilizes an Object-Oriented Design (OOD) approach mapped to a relational database scheme. The architectural pattern follows standard Model-View-Controller (MVC)

principles adapted for a RESTful API architecture. Routes map to controllers, which interact with Prisma models to execute business logic.

3. Architectural and Component-Level Design

3.1 System Structure

The system utilizes a three-tier client-server architecture. The presentation layer (HTML/CSS) communicates asynchronously with the Node.js application layer. The application layer processes business rules and delegates data persistence to the PostgreSQL database via the Prisma ORM abstraction layer.

3.1.1 Architecture Diagram

The following logical architecture represents the three main tiers of the system:

```
[Client Browser (HTML/CSS)] ←(HTTP/REST)→ [Node.js / Express API]
                               ←(Prisma ORM)→ [PostgreSQL Database]
```

The Design Class Diagram (DCD), which provides a full object model of the system, is included in Appendix A.

3.2 Description for Components

The system is decomposed into four main components, each encapsulating a specific domain of functionality.

3.2.1 Component 1: Account & Profile Management

Processing Narrative (PSPEC):

Handles user registration with interest selection, JWT token generation, secure password hashing via bcrypt, and profile updates. It also manages the many-to-many relationship of user interests and the follow/unfollow interactions between users. This component is responsible for the Unauthenticated User actor gaining access to the system (UC001).

Interface Description:

Endpoint	Method	Inputs	Outputs
/api/register	POST	username, email, password, interests[]	HTTP 201 + redirect to login / HTTP 409 Conflict
/api/login	POST	email, password	HTTP 200 + JWT Token / HTTP 401
/api/profile	PUT	JWT, updated fields	HTTP 200 + updated User JSON
/api/users/:id/follow	POST	JWT	HTTP 200 / HTTP 404

Interaction Diagram:

Refer to Appendix B – Figure B.1 (UC001: Register with Interest Selection) for the sequence diagram of the registration flow.

3.2.2 Component 2: Content Management

Processing Narrative (PSPEC):

Manages the full lifecycle of posts, including text content validation, image parsing via the Multer middleware, and hashtag extraction from post content. Handles linking likes and comments to specific posts and supports deletion of owned posts.

Interface Description:

Endpoint	Method	Inputs	Outputs
/api/posts	POST	JWT, content (text), image (optional, multipart)	HTTP 201 + Post object
/api/posts/:id	DELETE	JWT	HTTP 200 / HTTP 403 Forbidden
/api/posts/:id/like	POST	JWT	HTTP 200
/api/posts/:id/comments	POST	JWT, text	HTTP 201 + Comment object

3.2.3 Component 3: Recommendation Engine

Processing Narrative (PSPEC):

A rule-based engine that collects a user's selected interests and their historical interaction signals (hashtags derived from liked and commented posts). It queries non-followed posts from the database and scores them based on hashtag match count. The top 10 highest-scored posts are returned as recommendations. This component is strictly rule-based and does not employ machine learning, in accordance with the project scope.

Interface Description:

Endpoint	Method	Inputs	Outputs
/api/recommendations	GET	JWT Token (userId extracted server-side)	HTTP 200 + JSON array of top-10 Post objects

Interaction Diagram:

Refer to Appendix B – Figure B.2 (UC008: Content Recommendation) for the sequence diagram.

3.2.4 Component 4: Messaging & Notifications

Processing Narrative (PSPEC):

Handles the creation and retrieval of private, direct textual messages between two registered users. The system verifies that the recipient exists before inserting the message record. Conversation history is returned sorted in ascending timestamp order. Notification records are automatically generated by other components upon follow, like, or comment events.

Interface Description:

Endpoint	Method	Inputs	Outputs
/api/messages	POST	JWT, recipient_id, content	HTTP 201 + Message object / HTTP 404 (recipient not found)
/api/messages/:partner_id	GET	JWT	HTTP 200 + JSON list of Messages (sorted ascending)

Interaction Diagram:

Refer to Appendix B – Figure B.3 (UC009: Direct Messaging) for the sequence diagram.

3.3 Component Processing Detail & Design Class Hierarchy

Based on the UML Design Class Diagram (see Appendix A), the system implements the following object models:

1. User Class

Item	Detail
Attributes	+int id, +String username, +String email, +String password_hash, +String bio, +timestamp created_at
Operations	+register(username, email, password, interests), +login(email, password), +updateProfile(fields), +follow(targetUser), +unfollow(targetUser)
Relationships	Has many Post, Comment, Like, Message, and Notification objects. Many-to-many with Interest.

2. Post Class

Item	Detail
Attributes	+int id, +int user_id, +String content, +String image_path, +timestamp created_at
Operations	+create(content, imageFile), +delete(), +like(), +unlike(), +addComment(text), +deleteComment(id)
Relationships	Contains many Comment and Like objects. Tagged with many Hashtag objects.

3. RecommendationEngine Class

Item	Detail
Attributes	+int userId, +List<String> interestTags, +List<String> interactionTags
Operations	+collectSignals(userId), +scorePost(post), +recommend(userId)
Relationships	Analyzes User; operates on Post objects.

4. Supporting Classes

Class	Key Attributes / Methods
Message	+int id, +int sender_id, +int recipient_id, +String content +send(), +getConversation()
Notification	+int id, +int user_id, +int actor_id, +String type, +boolean is_read
Like	Relational entity – connects User to Post with timestamp

Class	Key Attributes / Methods
Comment	Connects User to Post with text content and timestamp
Interest	+int id, +String name – core categorization for registration
Hashtag	+int id, +String name – categorization entity extracted from post content

3.4 Dynamic Behavior

The dynamic behavior of the system is documented through sequence diagrams for each primary use case. These diagrams are provided in Appendix B. The three key interaction flows are:

- UC001 – Register with Interest Selection: Demonstrates the multi-step registration flow including uniqueness validation, bcrypt hashing, and user-interest association insertion.
- UC008 – Content Recommendation: Demonstrates the recommendation engine's signal collection (interests + interaction hashtags), non-followed post querying, and rule-based scoring and ranking.
- UC009 – Direct Messaging: Demonstrates both the message-sending flow (with recipient validation) and the conversation-reading flow.

4. Restrictions, Limitations, and Constraints

Restriction	Description	Impact
Stateless JWT	JWTs cannot be revoked server-side before expiry. The system relies on short expiration windows and client-side token deletion on logout.	Security risk on token theft; no server-side session invalidation.
Local Filesystem Storage	Media files are stored locally, meaning horizontal scaling of the Node.js backend across multiple instances will require migrating to external storage.	Scalability bottleneck in future deployment.
Rule-Based Recommendation	The recommendation engine's performance may degrade linearly as total system posts and hashtags grow.	Requires database indexing on foreign keys and hashtag columns for acceptable performance at scale.
No Real-Time Messaging	Messaging is REST-based only; users must manually refresh to receive new messages. WebSockets are explicitly out of scope.	Reduced UX for active messaging; no live notifications.
Plain HTML/CSS UI	JavaScript frameworks are prohibited, limiting the interactivity and responsiveness of the frontend.	The UI cannot perform client-side routing or dynamic DOM updates without full page reloads.

5. Conclusion

This Software Design Specification outlines a robust, RESTful architecture tailored to the exact requirements of the CSE3044 Term Project. By leveraging Node.js, Express.js, and Prisma ORM, the design strictly adheres to the technological constraints while providing a highly structured object-oriented implementation.

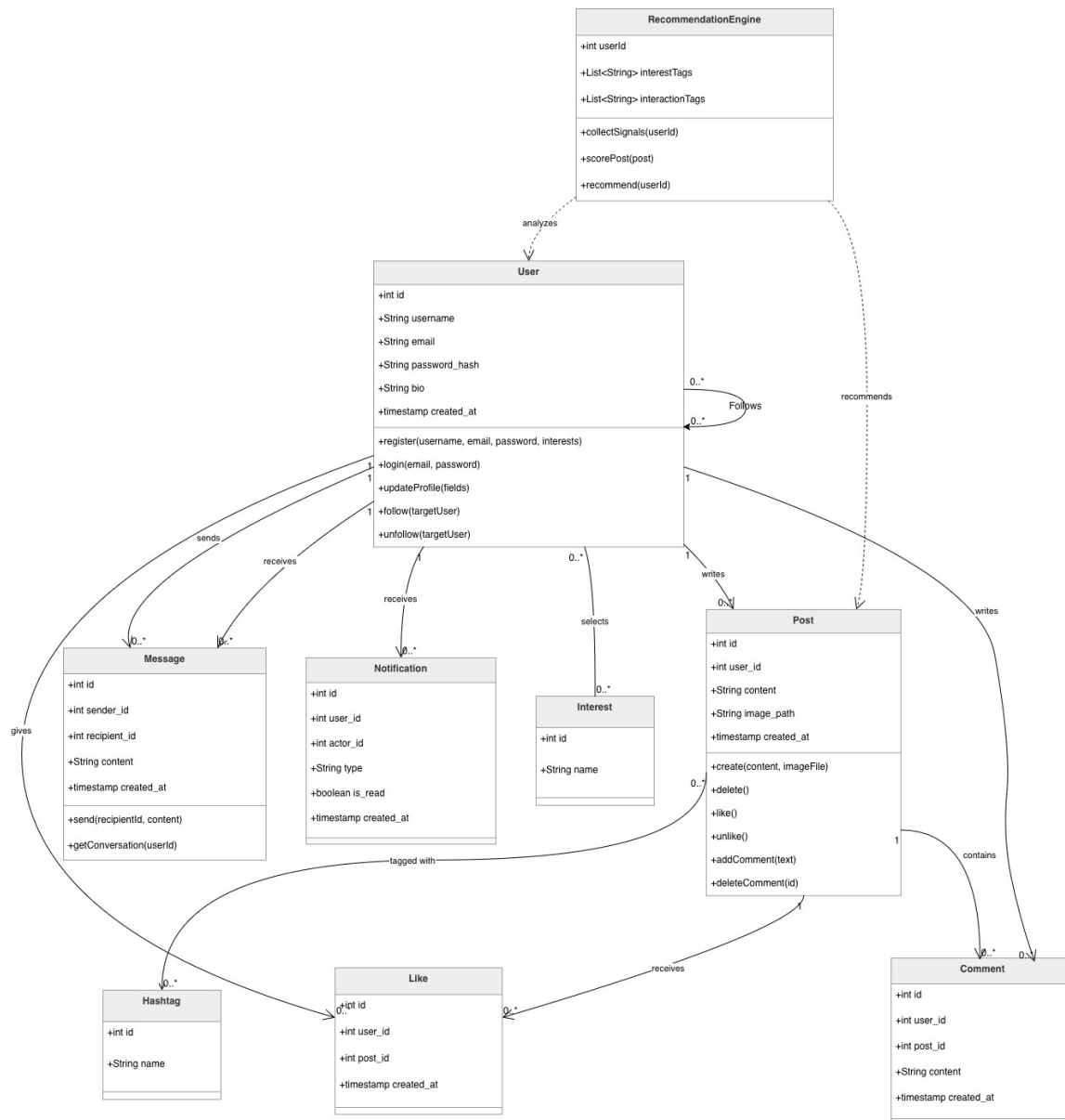
The logical segregation of four components (Account & Profile Management, Content Management, Recommendation Engine, and Messaging & Notifications) ensures modularity, making the plain HTML/CSS frontend integration straightforward and maintaining clear boundaries for business logic.

All sequence diagrams and the Design Class Diagram are appended in the Appendices section for reference. The design remains within the defined project scope, with the rule-based recommendation engine specifically implemented to avoid the explicitly out-of-scope machine learning approach.

Appendix A – Design Class Diagram (DCD)

Figure A.1 below presents the full Design Class Diagram (DCD) of the Social Network system, illustrating all classes, attributes, operations, and relationships.

Figure A.1: Design Class Diagram (DCD)



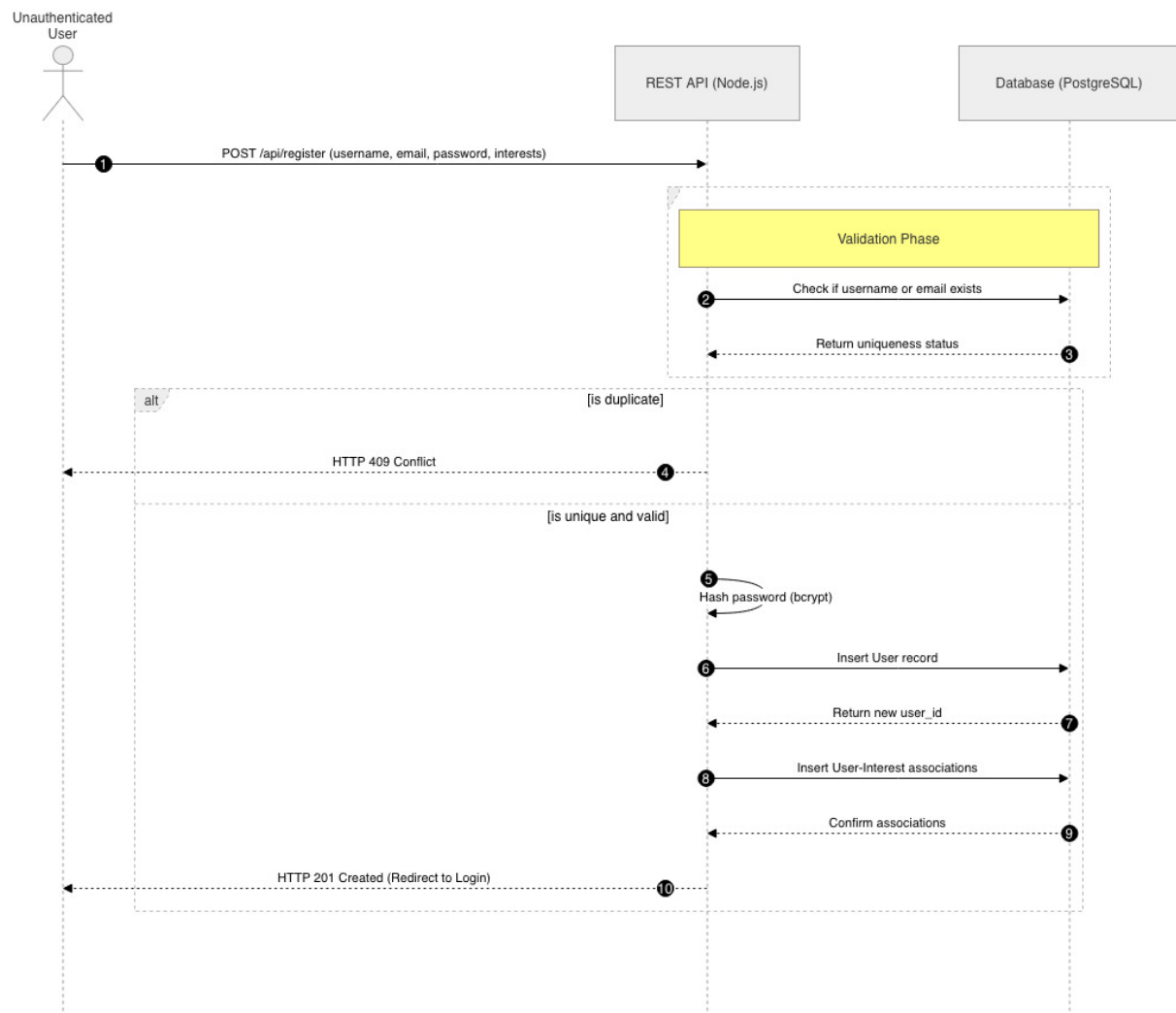
Appendix B – Sequence Diagrams

This appendix contains the UML sequence diagrams for the primary use cases of the Social Network system.

B.1 - Register with Interest Selection

This sequence diagram illustrates the registration flow: from the initial POST /api/register request through uniqueness validation, bcrypt password hashing, user record insertion, and user-interest association creation, ending with an HTTP 201 redirect to login. The alternate flow (HTTP 409 Conflict) covers the case where a username or email already exists.

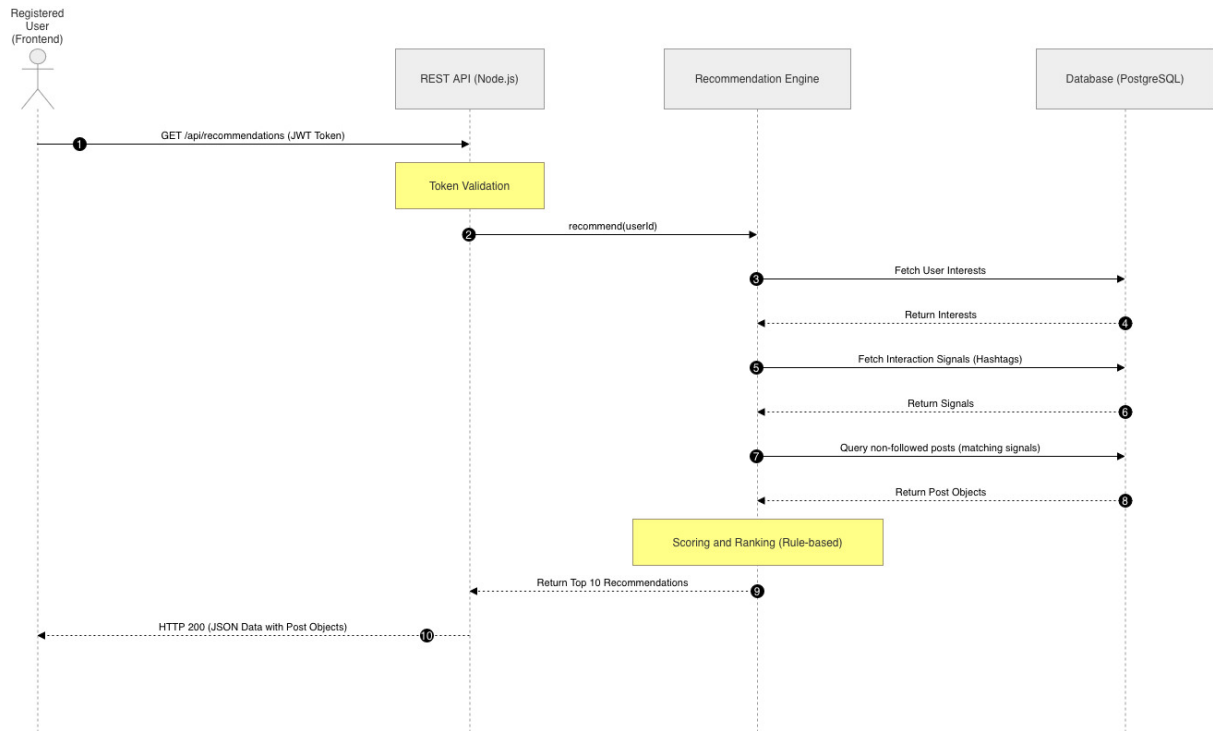
Figure B.1: Register with Interest Selection



B.2 - Content Recommendation

This sequence diagram illustrates the content recommendation flow: The Registered User triggers GET /api/recommendations. After token validation, the REST API delegates to the Recommendation Engine, which fetches user interests and interaction signals (hashtags) from the database, queries non-followed posts matching those signals, applies rule-based scoring and ranking, and returns the top 10 recommendations to the API, which delivers them as HTTP 200 JSON.

Figure B.2: Content Recommendation



B.3 - Direct Messaging

This sequence diagram covers two flows. The Sending a Message flow shows: POST /api/messages with recipient_id, content, and JWT; token validation; recipient existence check (HTTP 404 if not found); message insertion into the database; and HTTP 201 Created. The Reading a Conversation flow shows: GET /api/messages/:partner_id with JWT; token validation; query of messages sorted ascending by timestamp; and HTTP 200 with a JSON list of messages.

Figure B.3: Direct Messaging

